OME-Arrow ↦ Unifying Images, Metadata, and Features in an Interoperable Data Model for High-Content Imaging

Dave Bunten[1], Jenna Tomkinson[1], Michael Lippincott[1], Cameron Mattson[1], Julia B. Curd[1], Gregory P. Way[1]

[1]*Department of Biomedical Informatics, University of Colorado Anschutz Medical Campus*

I. Why OME-Arrow?

	A	B	C
1	ImageNumber	ObjectNumber	Metadata_FileLocation
2	1	1	file:/app/inputdata/nuclei1_out_c00_dr90_image.tif
3	1	2	file:/app/inputdata/nuclei1_out_c00_dr90_image.tif
4	1	3	file:/app/inputdata/nuclei1_out_c00_dr90_image.tif

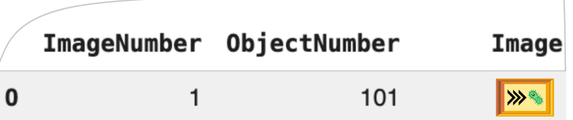

Figure 1: Modern bioimaging workflows depend on connecting **images**, **metadata**, and **derived measurements**. When these are split across disconnected files and systems, analysis is harder to join, reproduce, and scale. **OME-Arrow** provides a linked, queryable data model for these components in code-first and SQL-first analytical workflows.

II. An interoperable image data model

	ImageNumber	ObjectNumber	Image
0	1	101	
1	1	102	
2	2	201	

Figure 2: By representing image content as **Arrow-compatible structures** alongside metadata and features, OME-Arrow enables: **Explicit relationships** between pixels, metadata, and derived features, **Direct joins and filtering** in SQL and DataFrame workflows, **Cross-language interoperability** through Arrow-native representations, **Standards alignment** with the broader open bioimaging ecosystem

III. Benchmarks for OME-Arrow

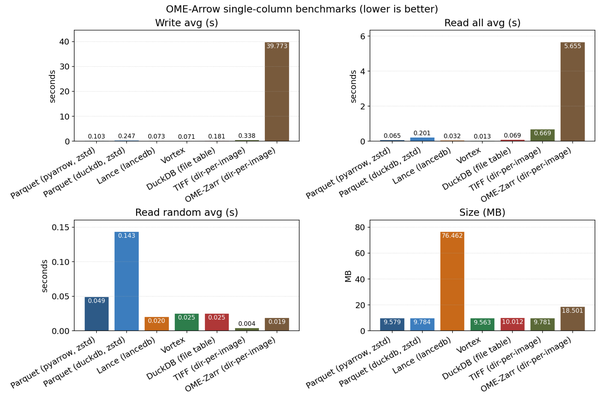

Figure 3: Current benchmark results suggest operation-dependent tradeoffs rather than one universal winner. In ome-arrow-benchmarks, OME-Zarr can perform strongly for sparse random image access patterns, while Arrow-table-native layouts provide practical advantages for table-centric join/filter workloads and broader analytical interoperability. Lance shows competitive random-access behavior in OME-Arrow-oriented tests, making it a strong candidate for large image-linked table repositories.

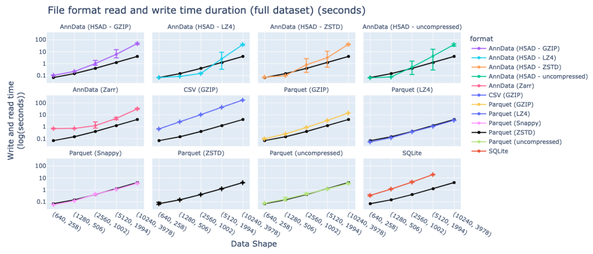

Figure 4: In CytoTable-benchmarks, Parquet and other Arrow-compatible tabular workflows generally scale better and run faster at larger data volumes than Zarr- and AnnData-oriented table paths, supporting the use of Arrow-native tables for large-scale profiling analytics.

IV. Quick technical demonstration

```python
# install with `pip install ome_arrow`
from ome_arrow import OMEArrow

# Create an OME-Arrow object from an OME-Zarr
oa = OMEArrow("image.ome.zarr")

# Export to Parquet directly
oa.export(how="ome-parquet", out="image.parquet")

# Create and collect a "lazy" crop of an image
lazy = OMEArrow.scan("image.parquet").slice_lazy(0, 512, 0,
512).collect()

# vizualize the image using pyvista for jupyter-friendly
views
oa.view(how="pyvista)
```

V. Full Cytomining integration

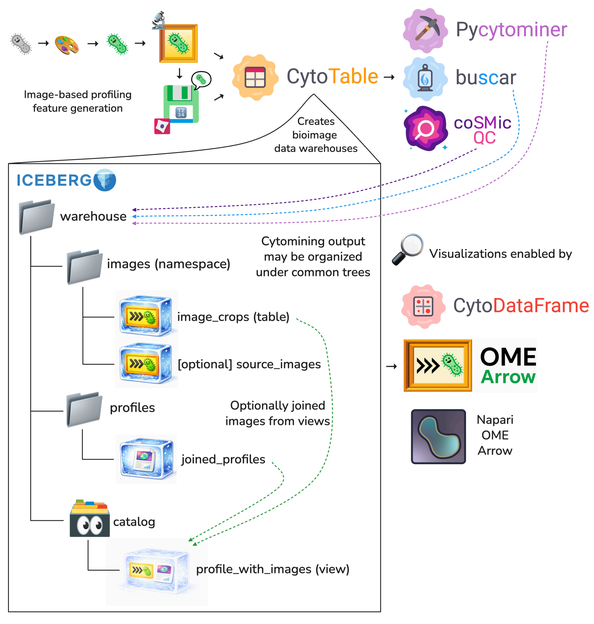

Figure 5: OME-Arrow complements Cytomining tools in a modular stack: OME-Arrow provides built-in visualization (`matplotlib`, `pyvista`) for direct inspection; `napari-ome-arrow` adds interactive viewing for OME-Arrow/OME-Parquet data; `CytoDataFrame` supports DataFrame-centered analysis of image-linked features and metadata; `coSMicQC` provides quality control with image context; `buscar` enables heterogeneity-aware single-cell compound ranking; and `pycytominer` supports profiling and normalization workflows.

VI. OME-Zarr and iceberg-bioimage

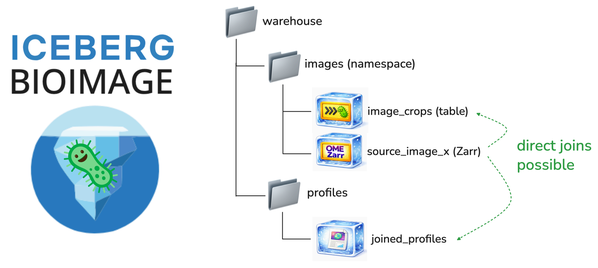

Figure 6: OME-Zarr is strong for cloud-native image storage and distribution. OME-Arrow is complementary (not a replacement) when images must be queried with tabular metadata and measurements. One integration pattern uses `iceberg-bioimage` as a warehouse/control-plane layer and `duckdb_zarr` for analytical access to OME-Zarr-backed data. Here, **OME-Zarr / OME-TIFF** remain exchange formats, while **OME-Arrow / Parquet / Arrow-native tables** support joins and analytics, with **Lance** as a random-access table option.

VII. In practice: ALSF pediatric cancer workflows

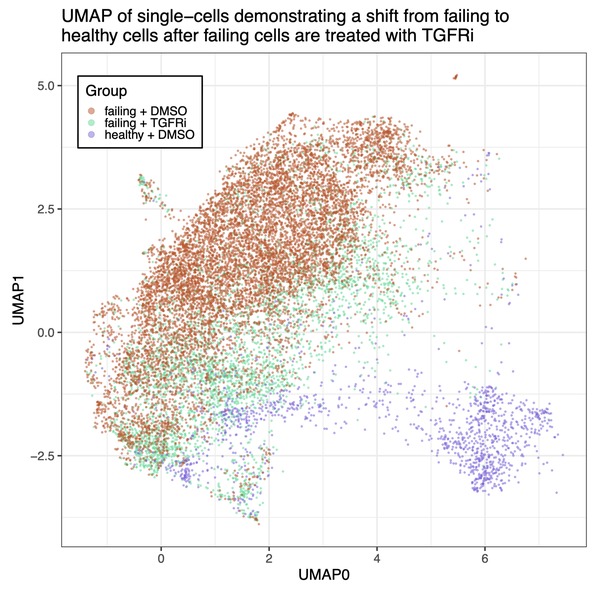

Figure 7: This framework is being used with **Alex's Lemonade Stand Foundation (ALSF)** data to support integrated image-based profiling. Instead of keeping raw images, extracted features, and experimental metadata in disconnected stores, OME-Arrow enables one linked representation that can be queried, audited, and reused across teams: **end-to-end context retention** (image pixels, per-cell features, and perturbation metadata stay connected), **faster biological iteration** (rank and compare candidate phenotypes across treatments and sample backgrounds without manual file stitching), **more reproducible pipelines** (explicit links between image records and derived tables reduce ambiguity during QC and re-runs), **operational scalability** (the same workflows run from local notebooks to larger repository and warehouse settings), and **Cytomining interoperability** (OME-Arrow + CytoDataFrame + coSMicQC + pycytominer + buscar provide a coherent path from ingest to profiling, QC, and hit prioritization). Practical ALSF outcome: teams can more quickly connect cellular morphology changes to experimental conditions in pediatric cancer studies.

VIII. Acknowledgements

We thank those who have inspired, contributed, or helped support OME-Arrow and the broader open bioimaging ecosystem:

- Open source science from the **Open Microscopy Environment**
- The communities behind **Apache Arrow**, **Apache Iceberg**, **napari**, and Cytomining ecosystem projects
- Special thanks to the **napari community** for open, collaborative development around interactive bioimage analysis
- Members of the **Way Lab** at the University of Colorado Anschutz Medical Campus
- **Department of Biomedical Informatics** within the School of Medicine at the University of Colorado Anschutz Medical Campus